Rastera

Rastera is a small language with symbolic control flow: `@` defines a function, `?/:` is if/else, `*` is a while loop and `^` returns. This repository implements it twice, as the two phases of the course project:

Phase	Program	What it does
1	`rasteri`	Flex + Bison interpreter. Parses a `.rst` file, checks it, and runs `@main` directly. No IR, no executable is produced.
2	`rasterc`	Flex + Bison ahead-of-time compiler. Builds an AST, runs semantic analysis, emits LLVM 18 IR and links a native Linux executable.

Both programs accept the same language and are checked by the same test cases (`tests/pass`, `tests/fail`). The normative language rules are in `docs/LANGUAGE_SPEC.md`; the original project proposal is `docs/proposal.pdf`.

```
@main() [
    print(6 * 7);
    ^ 0;
]
```

1. Requirements (Linux)

Primary path: Docker. The whole toolchain lives in one Docker image (`Dockerfile`, Ubuntu 24.04), so nothing else has to be installed. `./dev.sh` builds that image on first use and runs every command inside it. Only `docker build` and `docker run` are used, so the Docker Compose plugin is not needed.

On Ubuntu or Debian, install Docker Engine from the distribution package, and git to clone:

```
sudo apt-get update
sudo apt-get install -y docker.io git
sudo usermod -aG docker "$USER"     # then log out and back in so `docker` works without sudo
docker run --rm hello-world         # optional check that Docker works
```

(The development machine used for this README runs Ubuntu 24.04 with the `docker.io` package, Docker 29.1.3, and a user that is a member of the `docker` group.) For other distributions follow the Docker Engine installation guide for your distribution on docs.docker.com. Under Windows use a WSL2 Ubuntu terminal with Docker available inside it; the commands below are for Linux.

Optional native path (no Docker). On Ubuntu 24.04 install the same packages the `Dockerfile` installs:

```
sudo apt-get install -y build-essential cmake ninja-build git flex bison \
    llvm-18-dev clang-18 libzstd-dev zlib1g-dev libedit-dev libcurl4-openssl-dev
```

The build requires: CMake >= 3.20 (24.04 ships 3.28.3), Ninja, a C and C++17 compiler with `cc` available for linking, Flex >= 2.6 (2.6.4), Bison >= 3.8 (3.8.2) and the LLVM 18 development package (`llvm-18-dev`). Native configure and build are the same two commands `dev.sh` runs:

```
cmake -S . -B build -G Ninja
cmake --build build
```

These commands were verified inside the Docker image on a fresh `git clone`. The wrapper `./dev.sh` is the supported route; if CMake cannot find LLVM natively, add

`-DLLVM_DIR=/usr/lib/llvm-18/lib/cmake/llvm` to the configure command.

2. Get, build and test

```
git clone https://github.com/MHassanAbaid24/Rastera.git
cd Rastera
./dev.sh build        # builds the image on first use, then build/rasteri and build/rasterc
./dev.sh test         # build, then run BOTH test suites
```

`./dev.sh test` runs `tests/run_tests.sh build/rasterc` (every program compiled by `rasterc` and executed) and `tests/run_tests.sh --interp build/rasteri` (every program run through `rasteri`). Expected final output (one line per suite):

```
86 passed, 0 failed
86 passed, 0 failed
```

A failing case prints `FAIL <test>: <reason>` and the suite exits non-zero. `./dev.sh` with no arguments opens a shell in the container, and `./dev.sh <command>` runs any command there. All paths in this README are relative to the repository root, which is `/work` in the container. Build outputs go to `build/`, which git ignores.

3. Phase 1: the interpreter `rasteri`

Build: `./dev.sh build` produces `build/rasteri` (CMake target `rasteri`, sources `src/interp/interp.y` and `src/interp/lexer.l`; its Flex/Bison symbols use the prefix `ip` so it can live beside the compiler's parser).

Run:

```
./dev.sh build/rasteri examples/recursion_arrays.rst
```

- The program's `print(x)` calls write to stdout, one value and a newline each.
- Static errors (syntax errors, undefined variables, ...) are reported on stderr as `FILE:LINE: error: message` and nothing runs: exit status 1.
- A runtime error stops the program with `rastera: runtime error: MESSAGE` on stderr, exit status 2. Output printed before the error is kept.
- Otherwise the exit status is the value returned by `@main` (its low 8 bits, as for any Linux process).
- `rasteri` never writes an executable and never builds IR. Wrong usage prints `usage: rasteri <file.rst>` and exits 1; a missing file prints `FILE: No such file or directory`, exit 1.

4. Phase 2: the compiler `rasterc`

Build: `./dev.sh build` produces `build/rasterc` (and the runtime library `build/librastera_rt.a` that every program links with).

```
./dev.sh build/rasterc examples/recursion_arrays.rst -o build/recursion_arrays   # compile
./dev.sh build/recursion_arrays                                                   # run
```

Without `-o` the executable is named `a.out`. Stages can be inspected instead of building:

```
./dev.sh build/rasterc FILE.rst --tokens   # lexer output only
./dev.sh build/rasterc FILE.rst --ast      # parse only, print the AST
./dev.sh build/rasterc FILE.rst --ir       # stop after code generation, print LLVM IR
```

- Exit status of `rasterc` itself: 0 on success; 1 for a missing file, bad usage or any static error (`FILE:LINE: error: message` on stderr, no executable written). Usage text: `usage: rasterc <file.rst> [-o <output>] [--tokens | --ast | --ir]`.
- `--ast` stops after parsing, so it also prints programs that semantic analysis would reject (see `examples/static_error.rst`).
- The produced executable is a normal Linux program: `print` writes to stdout, the exit status is `@main`'s result, and a runtime error prints `rastera: runtime error: MESSAGE` on stderr and exits 2.

Real inspection output for `tests/pass/return_literal.rst` (`@main() [ ^ 42; ]`):

```
$ ./dev.sh build/rasterc tests/pass/return_literal.rst --tokens
3: '@'
3: "identifier" main
3: '('
3: ')'
3: '['
4: '^'
4: "integer" 42
4: ';'
5: ']'

$ ./dev.sh build/rasterc tests/pass/return_literal.rst --ast
[1] Program
  [3] Func main
    [3] Block
      [4] Return
        [4] Int 42

$ ./dev.sh build/rasterc tests/pass/return_literal.rst --ir
; ModuleID = 'tests/pass/return_literal.rst'
source_filename = "tests/pass/return_literal.rst"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

define i32 @rs_main() {
entry:
  ret i32 42
}
```

5. Examples

The four programs in `examples/` were run on `rasteri` and, where they are valid programs, compiled with `rasterc` and executed. The output below is what those runs printed.

5.1 Recursion and arrays: `examples/recursion_arrays.rst`

```
# Recursion and arrays: fill an array with Fibonacci numbers computed
# recursively, then compare one element with a literal array.
@fib(n) [
    ? (n < 2) [
        ^ n;
    ]
    ^ fib(n - 1) + fib(n - 2);
```

```
]

@main() [
    expected := <0, 1, 1, 2, 3, 5, 8, 13>;
    results := arr(8);
    i := 0;
    * (i < 8) [
        results[i] := fib(i);
        i := i + 1;
    ]
    print(results[7]);
    print(results[7] == expected[7]);
    ^ 0;
]
```

```
$ ./dev.sh build/rasteri examples/recursion_arrays.rst
13
1
(exit status 0)

$ ./dev.sh build/rasterc examples/recursion_arrays.rst -o build/recursion_arrays
$ ./dev.sh build/recursion_arrays
13
1
(exit status 0)
```

5.2 While loop and logic: `examples/while_logic.rst`

```
# While loop and logic: sum the numbers 1..20 that are divisible by 3 or 5,
# then show that & and | short-circuit.
@main() [
    i := 1;
    total := 0;
    * (i <= 20) [
        ? (i % 3 == 0 | i % 5 == 0) [
            total := total + i;
        ]
        i := i + 1;
    ]
    print(total);

    zero := 0;
    # The right side of & would divide by zero, but it is never evaluated.
    print(zero != 0 & 10 / zero > 1);
    # The right side of | is skipped because the left side is already true.
    print(1 | 10 / zero);
    print(!zero);
    ^ 0;
]
```

```
$ ./dev.sh build/rasteri examples/while_logic.rst
98
0
1
1
(exit status 0)

$ ./dev.sh build/rasterc examples/while_logic.rst -o build/while_logic
$ ./dev.sh build/while_logic
```

```
98
0
1
1
(exit status 0)
```

5.3 Runtime error: `examples/runtime_error.rst`

```
# Runtime error: valid output first, then an out-of-range array read.
@main() [
    values := <10, 20>;
    print(values[1]);
    ^ values[2];
]
```

Run each binary directly. `rasteri` interprets the file and stops at the error; it produces no executable. `rasterc` compiles successfully (the error can only be found while the program runs), and the executable fails the same way when started:

```
$ ./dev.sh build/rasteri examples/runtime_error.rst
20
rastera: runtime error: array index out of range
(exit status 2; the error text is on stderr)

$ ./dev.sh build/rasterc examples/runtime_error.rst -o build/runtime_error
$ ./dev.sh build/runtime_error
20
rastera: runtime error: array index out of range
(exit status 2; the error text is on stderr)
```

5.4 Static error: `examples/static_error.rst`

```
# Static error: 'missing' is never assigned, so no program is run or built.
@main() [
    print(missing);
    ^ 0;
]
```

```
$ ./dev.sh build/rasteri examples/static_error.rst
examples/static_error.rst:3: error: undefined variable 'missing'
(exit status 1)

$ ./dev.sh build/rasterc examples/static_error.rst -o build/static_error
examples/static_error.rst:3: error: undefined variable 'missing'
(exit status 1; build/static_error is not created)
```

6. Language overview

Normative rules, with every error case, are in `docs/LANGUAGE_SPEC.md`. In brief:

```
# comment to end of line
@add(a, b) [                  # function; parameters are i32
    ^ a + b;                  # return
]
```

```
@main() [                    # exactly one @main(); its result is the exit status
    x := 5;                  # := creates a variable, or reassigns a visible one
    ? (x > 3) [              # if
        x := x - 1;
    ] : ? (x == 0) [         # else if
        x := 100;
    ] : [                    # else
        x := 0;
    ]
    * (x > 0) [              # while: checks first, runs while non-zero
        x := x - 1;
    ]
    a := <1, 2, 3>;          # array literal
    b := arr(4);             # 4 zeros
    b[0] := a[2];            # element read and write
    print(add(b[0], 1));     # built-in print: value and newline on stdout
    ^ 0;
]
```

Core features implemented in both `rasteri` and `rasterc`:

- Two types, never written: `i32` (wrapping 32-bit signed integer) and `array` (a reference to a fixed-length heap array of `i32`; `b := a;` aliases, it does not copy).
- Arithmetic `+ - * / %` and unary `-`; `/` and `%` truncate toward zero.
- Comparisons `== != < <= > >=` (non-associative) producing 0 or 1; `&`, `|`, `!` on integers, where `&` and `|` short-circuit.
- Variables with lexical block scope; `if`, `else if`, `else`; `while`.
- Functions with parameters, recursion, mutual recursion and calls before the definition. Functions return `i32`; arrays cannot be passed to or returned from functions.
- Arrays: `<a, b, c>`, `arr(n)`, `a[i]`, `a[i] := e;`, bounds checked at run time.
- Built-in `print(x)`.
- Runtime errors (exit 2): division or modulo by zero, `-2147483648 / -1` and `% -1`, array index out of range, negative array size.
- Static errors (exit 1): syntax errors, bad characters, integer literals above 2147483647, undefined or misused variables, wrong function arity, duplicate functions or parameters, built-in names reused as names, array/integer mismatches, missing `@main()`.

Not implemented in this revision, although mentioned as future work in the specification: range loops (`* i :: a..b`) and the graphics built-ins (`canvas`, `rgb`, `pixel`, `export`, `line`, `rect`).

7. Exit status and error summary

Situation	`rasteri FILE`	`rasterc FILE -o OUT`**, then** `./OUT`
Success	stdout output, exit status = result of `@main`	`rasterc` exits 0; `./OUT` prints and exits with the result of `@main`
Static error	`FILE:LINE: error: ...` on stderr, exit 1	same message, exit 1, no `OUT` written
Runtime error	stdout so far, `rastera: runtime error: ...` on stderr, exit 2	`rasterc` exits 0; `./OUT` behaves as in the left column
Bad usage / missing file	usage or `FILE: No such file or directory`, exit 1	same, exit 1

8. Pipeline

```
Phase 1:  source.rst -> src/interp/lexer.l -> tokens -> src/interp/interp.y -> run @main
Phase 2:  source.rst -> src/parse/lexer.l -> tokens -> src/parse/parser.y -> AST
          -> src/sema/ -> checked AST -> src/codegen/ -> LLVM IR
          -> src/backend/ -> object file -> cc + runtime/rastera_rt.c -> executable
```

9. Regenerating README.pdf

`README.pdf` is generated from this file by a dependency-free script (Python 3 standard library only); no LaTeX or pandoc is needed. It is deterministic: the same `README.md` always produces byte-identical output.

```
python3 docs/build_readme_pdf.py           # rewrite README.pdf from README.md
python3 docs/build_readme_pdf.py --check   # exit 1 if README.pdf is out of date
```

10. Project file inventory

Every file tracked in the repository, by directory. Generated files (`build/`) and git-ignored files (`.gitignore` ignores `build/`, `a.out` and `*.ppm`) are not listed.

10.1 Repository root

File	Purpose
`README.md`	This document (source of `README.pdf`).
`README.pdf`	Generated PDF version of this document.
`CMakeLists.txt`	CMake build: targets `rasteri`, `rastera_rt` (runtime library) and `rasterc`; runs Flex and Bison.
`Dockerfile`	Development image: Ubuntu 24.04 with CMake, Ninja, gcc/g++, Flex, Bison, LLVM 18.
`dev.sh`	Runs a command in the container; `build` and `test` shortcuts.
`.gitattributes`	Forces LF line endings and marks `*.pdf` as binary.

10.2 `.github/`

File	Purpose
`.github/workflows/ci.yml`	Continuous integration: builds the image, builds both programs, runs both test suites.
`.github/pull_request_template.md`	Template for pull request descriptions.

10.3 `docs/`

File	Purpose
`docs/LANGUAGE_SPEC.md`	Normative language specification.
`docs/proposal.pdf`	Original project proposal.
`docs/build_readme_pdf.py`	Generator for `README.pdf` (Python 3 standard library only).

10.4 examples/

File	Purpose
examples/recursion_arrays.rst	Recursive Fibonacci stored in an array; prints 13 and 1.
examples/while_logic.rst	While loop with |, &, ! and short-circuiting; prints 98 0 1 1.
examples/runtime_error.rst	Prints 20, then fails with an array index error (exit 2).
examples/static_error.rst	Undefined variable; rejected before anything runs (exit 1).

10.5 runtime/

File	Purpose
runtime/rastera_rt.c	C runtime linked into every compiled program: the process entry main that calls rs_main, rt_print (print), rt_alloc (array allocation with a length header and the negative-size check) and rt_panic (prints the runtime error, exits 2).

10.6 src/ (Phase 2 compiler, rasterc)

File	Purpose
src/main.cpp	rasterc driver: argument parsing, runs the pipeline, --tokens, --ast, --ir.
src/diag.hpp	Diagnostics: collects and prints FILE:LINE: error: message.
src/parse/lexer.l	Flex lexer for the compiler.
src/parse/parser.y	Bison grammar for the compiler; builds the AST.
src/parse/frontend.hpp	Interface between driver, lexer and parser (parse, lexer_begin, token_name).
src/ast/ast.hpp	AST node types and the visitor interface.
src/ast/ast_printer.hpp	Printer behind --ast.
src/sema/sema.hpp	Semantic analyzer interface.
src/sema/sema.cpp	Scope, type and call checks over the AST.
src/sema/symbols.hpp	Symbol table and scope structures.
src/codegen/codegen.hpp	LLVM IR generator interface.
src/codegen/codegen.cpp	AST to LLVM IR, including runtime-check calls.
src/backend/backend.hpp	Object emission and linking interface.
src/backend/backend.cpp	LLVM module to object file, then link with the runtime into an executable.

10.7 src/interp/ (Phase 1 interpreter, rasteri)

File	Purpose
src/interp/lexer.l	Flex lexer for the interpreter (generated symbols prefixed ip).
src/interp/interp.y	Bison grammar, execution nodes, static checks, evaluator and main of rasteri.

10.8 tests/

File	Purpose
tests/run_tests.sh	Test runner: tests/run_tests.sh BIN (compiler) or --interp BIN (interpreter). Reads # expect-exit, # expect-stdout, # expect-error, # expect-runtime-error directives.

tests/pass/ holds 45 programs that must run (expected stdout and exit status in directives); tests/fail/ holds 41 programs that must be rejected or fail at run time. Each runs in both suites, giving the 86 per suite.

tests/pass/ files:

File	Checks
arithmetic_acceptance.rst	`(2 + 3) * -4 % 7` gives exit status 250.
arithmetic_left_assoc.rst	Subtraction is left-associative (exit 4).
arithmetic_precedence.rst	`*` binds tighter than `+` (exit 14).
array_alias.rst	`b := a;` aliases; writing `b[0]` changes `a[0]` (exit 99).
array_alloc.rst	`arr(5)`, element stores, printing elements.
array_literal.rst	Array literal and summing its elements (exit 6).
array_store_load.rst	Store then load an element (exit 7).
array_zero.rst	`arr(0)` is a valid empty array.
call_in_expr.rst	`print` used as an expression returns 0 (prints 42).
cmp_basic.rst	All six comparison operators (exit 6).
code_after_return.rst	Unreachable statements after `^` are harmless (exit 7).
combined_array_literal_comparison.rst	Indexing an array literal inside a comparison in `?` (exit 7).
combined_recursive_array_index.rst	Recursion creating an array per call (exit 6).
func_args_left_to_right.rst	Arguments evaluate left to right (prints 1, 2; exit 12).
func_call_before_definition.rst	Calling a function defined later (exit 12).
func_factorial.rst	Recursive factorial (exit 120).
func_fibonacci.rst	Recursive Fibonacci (prints 55).
func_general_return.rst	`sq(x) + sq(x + 1) - 1` with calls inside expressions (exit 24).
func_mutual_recursion.rst	Mutual recursion (prints 1, 0).
func_param_is_local.rst	Assigning a parameter does not change the caller's variable.
func_params_basic.rst	Parameters passed and used (exit 5).
if_branches_both_return.rst	`if`/`else` where both branches return (exit 15).
if_else.rst	`if` / `else` (exit 7).
if_elseif_chain.rst	`else if` chain; 300 becomes exit 44 (low 8 bits).
if_scope.rst	Variable declared in a branch is local; outer variable still usable (exit 10).
if_without_else.rst	`if` without `else` (exit 42).
implicit_return.rst	Reaching the end of a function returns 0.
interp_deep_recursion.rst	`fib(20)` through deep recursion (exit 109).
interp_local_array_alias.rst	Alias of a local array inside a function (prints 9).
interp_wraparound.rst	`2147483647 + 1` wraps to `-2147483648`.
logic_short_circuit.rst	`&` and `|` skip the right operand when decided (prints 3, 4; exit 3).
logic_truth_precedence.rst	Logical operator precedence and `!` (prints four 1s).
print_acceptance.rst	Basic `print` (prints 7).
print_multiple.rst	Several `print` calls in order (10, 20, 30).
return_literal.rst	Returned literal becomes the exit status (42).
rt_array_zero.rst	`arr(0)` is created without a runtime error (prints 5).
unary_minus.rst	`-(-7)` is 7 (exit 7).
var_basic.rst	Variable creation and use (exit 5).
var_inner_block_reassigns_outer.rst	A bare block reassigns a visible outer variable (exit 2).
var_inner_declared_fresh_after_block.rst	A name first created inside a block is new again afterwards (exit 4).
var_reassign_arith.rst	Reassigning with arithmetic (exit 7).
while_nested_multiplication.rst	Nested while loops (prints 6).

File	Checks
while_return_from_body.rst	^ inside a loop body returns from the function (exit 23).
while_sum_1_to_10.rst	Sum 1 to 10 with while (prints 55).
while_zero_iterations.rst	A false condition runs the body zero times (prints 0).

tests/fail/ files (expected stderr text in parentheses; static errors exit 1, runtime errors exit 2):

File	Checks
array_arith.rst	Array nested inside an array literal <1, <2, 3>> (array literal elements must be integers).
array_index_non_array.rst	Indexing an integer (cannot index a non-array value).
array_negative_size.rst	arr with a negative size (runtime: negative array size).
array_print_array.rst	print of an array (expects an integer argument).
array_type_mismatch.rst	Reassigning an integer variable with an array (type mismatch).
bad_character.rst	Unknown character $ (unexpected character '$').
call_arg_undefined.rst	print(y) with y never assigned (undefined variable 'y').
call_unknown_func.rst	Call to an undefined function (unknown function 'unknown_func').
call_wrong_arity_too_few.rst	print() (wrong argument count for 'print').
call_wrong_arity_too_many.rst	print(1, 2) (wrong argument count for 'print').
cmp_nonassoc.rst	Chained comparison a < b < c (syntax error).
func_array_argument.rst	Array passed as a function argument (function 'first' expects an integer argument).
func_builtin_name.rst	Function named print (using built-in name 'print' as a function name).
func_duplicate.rst	Two functions with one name (duplicate function 'main').
func_duplicate_param.rst	Repeated parameter name (duplicate parameter 'a').
func_main_with_params.rst	@main with parameters (@main() must not have parameters).
func_param_builtin_name.rst	Parameter named print (... as a parameter name).
func_vars_not_shared.rst	A function using another function's variable (undefined variable 'x').
func_wrong_arity.rst	Wrong number of arguments (expected 2, got 1).
if_branch_var_not_visible.rst	Variable created inside an if branch used after it (undefined variable 'branch_var').
interp_output_before_runtime_error.rst	Output 7 is kept, then runtime error negative array size.
literal_too_large.rst	Literal 2147483648 (integer literal out of range).
logic_array_not_operand.rst	! applied to an array (logical operand must be an integer).
logic_array_operand.rst	&/| applied to an array (logical operands must be integers).
no_main.rst	No @main (program has no @main() function).
rt_division_by_zero.rst	Division by zero after printing 7 (division by zero).
rt_division_int_min.rst	-2147483648 / -1 (integer division overflow).
rt_index_read_high.rst	Read past the end (array index out of range).
rt_index_read_negative.rst	Read at a negative index (array index out of range).
rt_index_store_high.rst	Store past the end (array index out of range).
rt_modulo_by_zero.rst	Modulo by zero (modulo by zero).
rt_modulo_int_min.rst	-2147483648 % -1 (integer modulo overflow).
rt_negative_array_size.rst	Negative arr size at run time (negative array size).
syntax_missing_semicolon.rst	Missing ; (3: error: syntax error, unexpected ']').

File	Checks
`syntax_unclosed_paren.rst`	Unclosed parenthesis (`syntax error`).
`var_builtin_name.rst`	Variable named `print` (`using built-in name 'print' as a variable name`).
`var_inner_not_visible.rst`	Block variable used after the block (`4: error: undefined variable 'y'`).
`var_self_reference_undeclared.rst`	`x := x + 1;` with no earlier `x` (`undefined variable 'x'`).
`var_undefined.rst`	Use of an unassigned variable (`undefined variable 'z'`).
`while_array_condition.rst`	Array as a `while` condition (`while condition must be an integer`).
`while_body_variable_not_visible.rst`	Loop-body variable used after the loop (`undefined variable 'hidden'`).